

ox-alpha's fixes, independently verified

Every fix the implementer claimed was re-read in the actual code of `rewrite/python@c7e0fee`, and the new money-path tests were read line-by-line — the same "trust nothing until read" discipline applied to `PARITY.md` in pass 1.

RE-AUDITED HEAD
c7e0fee

FIX COMMIT
74dba0e

FILES CHANGED
14

NEW MONEY TESTS
3

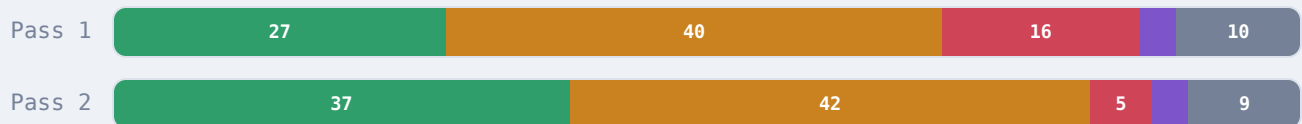
METHOD
static

✓ Accept tranche 1 — real, tested, safe to build on

55% → **64%**
WEIGHTED PARITY

The CRITICAL money-path block went from **0 of 4 working** to **≈3.5 of 4**: a customer can now pay via an online gateway and actually receive their config, and a wallet top-up credits instead of crashing. The response doc is candid and accurate. Two residuals remain (§3) and the branch is not yet production-ready — but this is a materially different, much healthier branch than pass 1.

01 Status movement



Done 27→37 Partial 40→42 Buggy 16→5 Risky 3→3 Missing 10→9

Eleven items left the buggy column: F-006/022/030/032/033 (the whole gateway/Stars settlement chain), F-016 (change-location), F-048/075 (lucky wheel + prize config), F-078 (disable-config commit), F-082 (node uptime). F-005 moved buggy → risky (ordering fixed, race open) and F-068 missing → partial (admin gating, level 1).

02 Independent verification

CRITICAL — money path

ID	CLAIM	VERDICT	EVIDENCE
----	-------	---------	----------

ID	CLAIM	VERDICT	EVIDENCE
C1	settle_wallet_topup restored	✓ confirmed + test	Defined <code>service.py:135</code> ; test asserts exactly-once credit + idempotent replay.
C2	gateway purchase sets invoice_id	✓ confirmed + test	<code>pay_gateway</code> writes pending invoice first; test drives real <code>settle()</code> , asserts invoice→enable, ledger empty .
C3	buyer DM + admin report	🕒 partial	Bot threaded for 5 gateways. Residual: cubepay's <code>settle_tuple</code> still notifies <code>None</code> & skips <code>_report</code> . Topic threading deferred.
C4	charge-first + atomic decrement	🕒 partial	Ordering fixed (charge→provision→refund). Residual: <code>change()</code> is read-modify-write, not atomic — concurrency double-spend still possible on PostgreSQL.

HIGH

ID	CLAIM	VERDICT	EVIDENCE
H1	changeloc preserves quota	✓ confirmed	Reads source config, carries remaining bytes/days, removes old.
H4	disableconfig commits	✓ confirmed	<code>await s.commit()</code> added at <code>builtin.py:88</code> .
H5	DB-backed admin gating (L1)	✓ confirmed	Reads <code>admin</code> table (60s cache); rule levels deferred (H5b).
H6	uptime_node decrypts	✓ confirmed	Decrypts via <code>_decrypt_secret</code> before building the adapter.
H2 · H3 · H7 · H8	deferred to tranche 2	— accurate	Not claimed fixed; pass-1 findings stand (manual_sell, reminder DMs, real migration, persistent state).

MEDIUM / LOW

ID	CLAIM	VERDICT	EVIDENCE
M2	gift code value + count	✓ confirmed	Real percent; count via <code>COUNT(*) GROUP BY</code> . Redemption increment with M1.
M3	wheel reads config	✓ confirmed	<code>_wheel_prizes</code> reads the setting, falls back to defaults.
M10	mini-app wallet inline settle	✓ confirmed	charge→provision→refund (402/502).
L1	dead/insecure code removed	✓ confirmed	Shadowed insecure <code>miniapp</code> , placeholder app, <code>_TOPUP_UNUSED</code> , <code>_expire_from</code> all gone.

ID	CLAIM	VERDICT	EVIDENCE
L4	boot refuses default secret	✓ confirmed	<code>server.py:38-41</code> SystemExit on empty token / default session secret.
M1 · M4-M9 · L2 · L3 · L5	deferred	— accurate	Honestly labelled; matches pass 1.

03 Residuals the response under-states

ACT ON THESE — THEY FINISH THE CRITICAL BLOCK PROPERLY

1. **C4 is not race-safe.** The provision-before-charge bug is genuinely fixed, but "atomic conditional decrement" is inaccurate — `WalletService.change` reads then writes in Python with no row lock. Two concurrent wallet buys can still double-spend on PostgreSQL. **Fix:** a single atomic `UPDATE "user" SET balance = balance - :amt WHERE id=:id AND balance >= :amt` (act on rowcount), or `SELECT ... FOR UPDATE` before the check.
2. **cubepay settlement is still silent.** `settle_tuple` (cubepay only) passes `bot=None` and skips `_report`. **Fix:** thread the bot into `settle_tuple` like `settle` does, and call `_report` there.

Test-suite check. The response reports "45 passed." I did not execute the suite (still a static pass) but read `tests/test_money_path.py` in full: the gateway-provision and top-up tests are **genuine** — they drive the real `settle()` and assert the criteria that matter (invoice activated, order paid, wallet untouched / credited once), and would fail on the pass-1 code. The third test only unit-checks the charge refusal — it does not exercise the full flow or the concurrency case. Net: honest tests, not gamed; the one missing dimension is exactly the C4 residual above.

04 What still stands between this and replacing main

- 1 **Finish CRITICAL:** close the C4 atomic-debit race and the cubepay notify/report gap — both small.
- 2 **Tranche-2 revenue items** (implementer's own list, agreed): customer expiry-reminder DMs, discount engine + redemption increments, referral commission & cashback, trial per-panel config, on-hold + `service_time` semantics, real DDL migration + settings view, persistent FSM/sessions.
- 3 **Tranche-3 breadth:** in-bot text customization and the rest of the admin surface, lottery/gift jobs, web-CRUD decision.

Dynamic pass (still owed): execute the 45 tests; each panel adapter against a real panel;

- 4 Alembic up/down on PostgreSQL; gateway callbacks against sandboxes; installer; white-label dispatch.

Re-audit pass 2 · verified statically against `rewrite/python@c7e0fee` · 2026-08-25. Every ✓ was re-read in the source; the two residuals in §3 were found by reading past the response's summary, not taken from it. Companion to the pass-1 report and PMO dashboard.